Unidad 5 — El DOM y el BOM

Interactuando con el navegador desde JavaScript

Materia: Programación 3 Frontend
Año: 2026
Docente: Lucas Ferrero

¿Qué es el DOM?

El Document Object Model (DOM)

Cuando el navegador carga un archivo HTML, no lo muestra directamente. Primero lo parsea (lo lee y analiza) y construye una **representación interna en memoria llamada DOM** (Document Object Model).

El DOM es una **interfaz de programación** que representa el documento HTML como un árbol de objetos. Cada etiqueta HTML se convierte en un **nodo** que JavaScript puede leer y modificar.

En otras palabras: el DOM es el puente entre tu HTML y tu JavaScript.

Características clave:

- Es una representación **viva**: si modificás el DOM, el navegador actualiza la pantalla en tiempo real.
- Es independiente del lenguaje, pero en el navegador se accede desde JavaScript.
- El punto de entrada siempre es el objeto global *document*.

Notas:

El DOM no es JavaScript. **Es una API que el navegador expone y a la que JavaScript tiene acceso**. Por eso *document*, *window*, *alert* no existen en Node.js: El DOM es del entorno del navegador, no del lenguaje.

El DOM como árbol de nodos

El DOM organiza el documento en una estructura jerárquica de árbol. Cada parte del HTML se convierte en un **nodo**.

Dado este HTML:

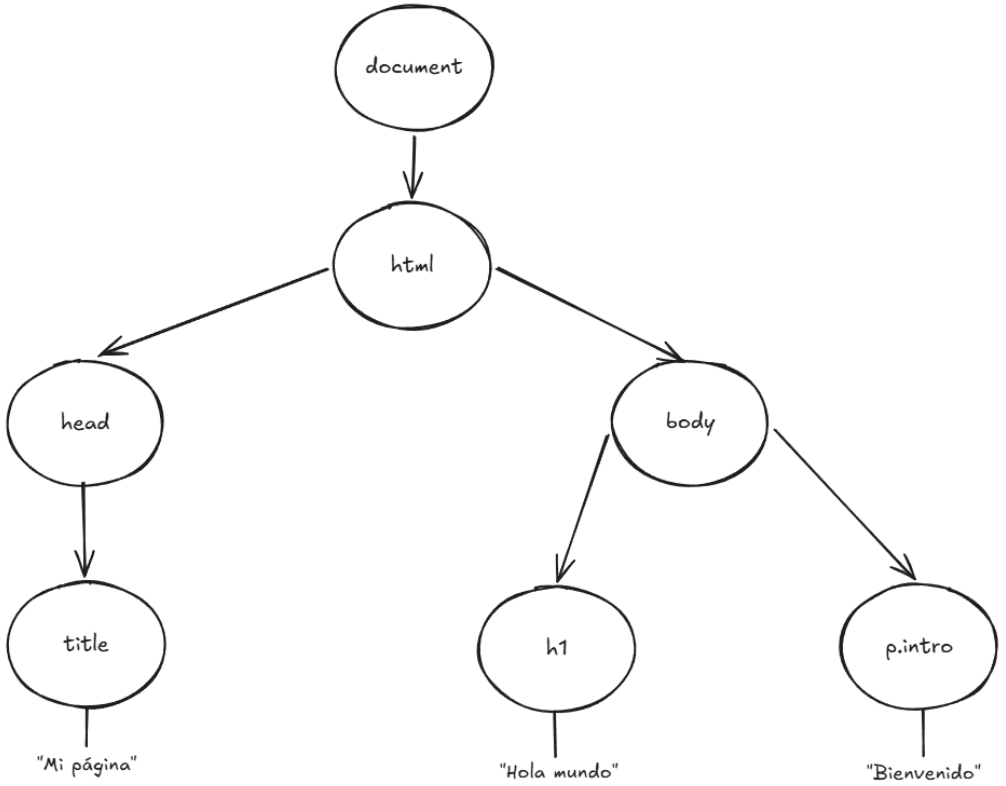
```
<!DOCTYPE html>
<html>
  <head>
    <title>Mi página</title>
  </head>
  <body>
    <h1>Hola mundo</h1>
    <p class="intro">Bienvenido</p>
  </body>
</html>
```

El árbol resultante es:

```
document
└── html
    ├── head
    │   └── title → "Mi página"
    └── body
        ├── h1 → "Hola mundo"
        └── p.intro → "Bienvenido"
```

Terminología clave:

- **Nodo raíz**: document
- **Nodo padre**: el que contiene a otro (body es padre de h1)
- **Nodo hijo**: el contenido directo de un nodo
- **Nodo hermano**: nodos al mismo nivel (h1 y p son hermanos)



Tipos de nodos en el DOM

No todo en el DOM es una etiqueta HTML. Existen distintos tipos de nodos:

Tipo	Descripción	Ejemplo
Element Node	Representa una etiqueta HTML	, <p>, <h1>
Text Node	El texto dentro de un elemento	"Hola mundo" dentro de un <p>
Attribute Node	Los atributos de un elemento	class="intro"
Document Node	El nodo raíz, punto de entrada	document
Comment Node	Comentarios HTML	<!-- comentario -->

Lo importante es entender que podemos obtener un Element Node, y que ese elemento tiene propiedades para acceder a su texto, sus atributos y sus nodos hijos.

El Browser Object Model (BOM)

El BOM representa todo lo que el navegador expone a JavaScript **más allá del documento HTML**. Su punto de entrada es el objeto global *window.*

A diferencia del DOM (que tiene un estándar oficial del W3C), el BOM no está completamente estandarizado, aunque los navegadores modernos lo implementan de forma muy similar.

Partes del BOM más usadas:

Objeto	Para qué sirve
window	El objeto global. Todo lo que existe en JS es propiedad de window
window.location	URL actual, redireccionar, recargar la página
window.history	Historial de navegación (atrás, adelante)
window.navigator	Info del navegador y sistema operativo
window.screen	Dimensiones de la pantalla
window.alert / confirm / prompt	Diálogos nativos del navegador
window.localStorage	Almacenamiento persistente en el navegador
window.setTimeout / setInterval	Temporizadores

```
// Ejemplos de BOM
console.log(window.location.href);    // URL actual
window.location.href = "https://google.com"; // redirigir
console.log(window.innerWidth);       // ancho del viewport
localStorage.setItem("usuario", "Lucas"); // guardar dato
```

Selección de elementos en el DOM

Para modificar el DOM, primero necesitamos obtener una referencia al elemento que nos interesa. JavaScript ofrece varios métodos para esto:

```
// Por ID (devuelve UN elemento o null)
const titulo = document.getElementById("titulo");

// Por clase (devuelve una HTMLCollection, no un array real)
const items = document.getElementsByClassName("item");

// Por etiqueta (devuelve una HTMLCollection)
const parrafos = document.getElementsByTagName("p");
```

getElementsByClassName y getElementsByTagName devuelven una HTMLCollection, que es similar a un array pero no es un array. No tiene métodos como .forEach() o .map()

querySelector y querySelectorAll

Estos métodos usan la misma sintaxis que los selectores de CSS, lo que los hace mucho más flexibles y expresivos.

```
// querySelector: devuelve EL PRIMER elemento que coincide (o null)
const titulo     = document.querySelector("h1");
const boton      = document.querySelector("#btn-enviar");
const destacado  = document.querySelector(".card.destacada");
const input      = document.querySelector("form input[type='email']");

// querySelectorAll: devuelve TODOS los elementos que coinciden
// Devuelve una NodeList (tiene forEach, pero no map/filter nativos)
const tarjetas   = document.querySelectorAll(".card");
const inputs     = document.querySelectorAll("input");

// Iterar con forEach ✅
tarjetas.forEach(tarjeta => {
  console.log(tarjeta.textContent);
});

// Convertir a Array real para usar map/filter ✅
const arrayTarjetas = Array.from(tarjetas);
// o con spread:
const arrayTarjetas2 = [...tarjetas];
```

Navegación entre nodos (Traversing)

Navegar el árbol del DOM: Una vez que tenemos un elemento, podemos movernos por el árbol del DOM desde este elemento:

```
const lista = document.querySelector("ul");

// Nodo padre
lista.parentElement;           // el elemento que contiene al ul

// Hijos
lista.children;                // HTMLCollection con los hijos directos
lista.firstElementChild;       // primer hijo
lista.lastElementChild;        // último hijo

// Hermanos
lista.nextElementSibling;      // el siguiente elemento hermano
lista.previousElementSibling;  // el anterior elemento hermano

// Ejemplo práctico: al hacer click en un li, resaltar su padre
const item = document.querySelector("li");
item.parentElement.style.border = "2px solid red";
```

Propiedad	Devuelve	Incluye texto/comentarios
children	HTMLCollection	❌ Solo elementos
childNodes	NodeList	✅ Todos los nodos
firstElementChild	Element	❌ Solo elementos
firstChild	Node	✅ Puede ser texto

DevTools y el DOM

El DevTools del navegador es la herramienta principal para trabajar con el DOM. Tabs más importantes:

Tab Elements:

- Visualizá el árbol del DOM en tiempo real.
- Editá HTML y CSS directamente.
- Seleccioná un elemento y accedé a él desde la consola con $0.

Tab Console:

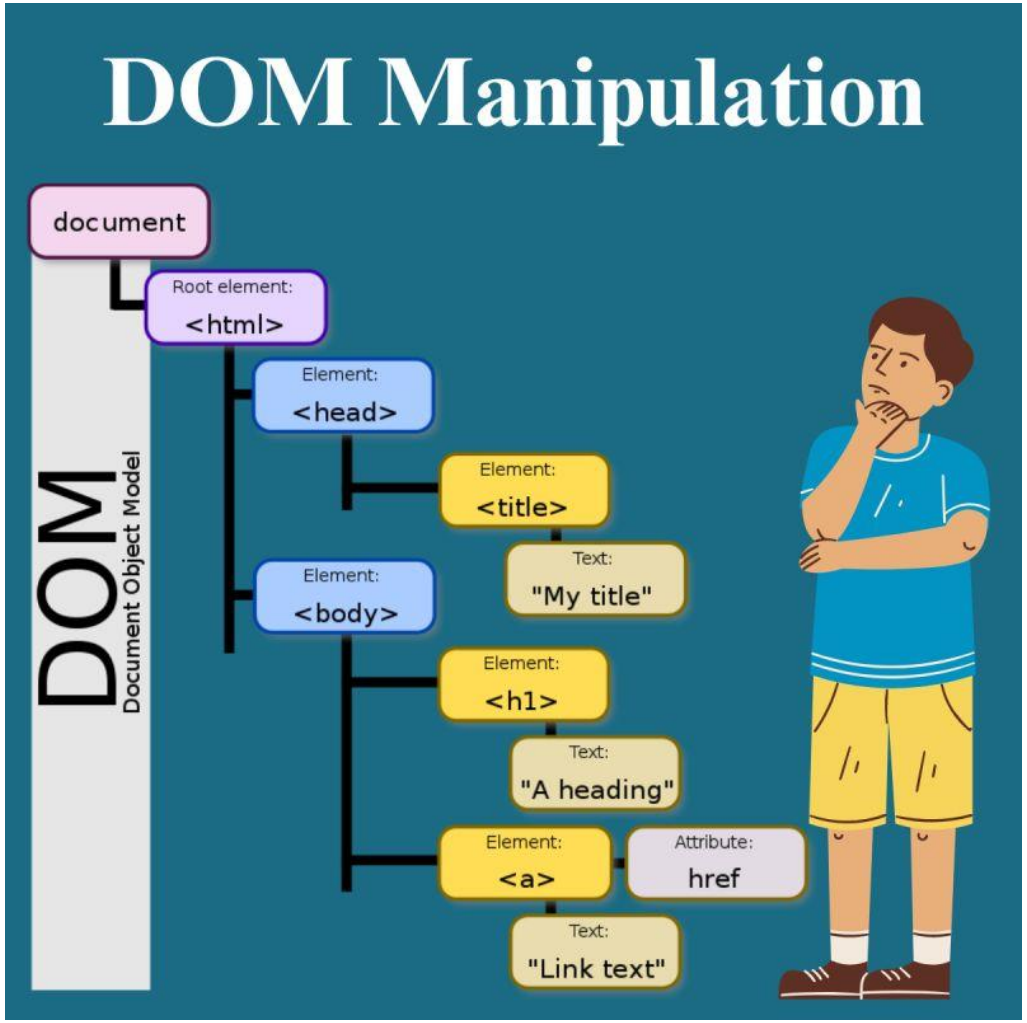
- Ejecutá JavaScript en tiempo real sobre la página.
- Usá document.querySelector(...) para experimentar.
- Los errores de JS aparecen aca.

```
// Atajos útiles en la consola del navegador
$0                  // el elemento actualmente seleccionado en Elements
$("h1")             // shortcut para querySelector (solo en DevTools)
$$(".card")         // shortcut para querySelectorAll (solo en DevTools)
console.dir($0)     // muestra todas las propiedades del nodo como objeto
```

Resumen de lo aprendido hasta el momento

Concepto	En una línea
DOM	Representación en memoria del HTML como árbol de objetos
BOM	APIs del navegador más allá del documento (window, location, etc.)
Nodos	Todo en el DOM es un nodo: elementos, texto, atributos
getElementById	Selecciona por ID, método clásico
querySelector	Selecciona el primer elemento que coincide con un selector CSS
querySelectorAll	Selecciona todos los elementos coincidentes (NodeList)
Traversing	Navegar entre nodos: padre, hijos, hermanos

Manipulación del DOM



Atributos del DOM

Los atributos HTML (id, class, src, href, disabled, etc.) se pueden leer y modificar desde JavaScript:

```
const imagen = document.querySelector("img");
const link   = document.querySelector("a");
const input  = document.querySelector("input");

// Leer atributo
imagen.getAttribute("src");          // "/img/foto.jpg"
imagen.getAttribute("alt");          // "Foto de perfil"

// Modificar atributo
imagen.setAttribute("src", "/img/nueva.jpg");
imagen.setAttribute("alt", "Nueva foto");

// Eliminar atributo
imagen.removeAttribute("alt");

// Verificar si existe un atributo
imagen.hasAttribute("alt");          // false (lo acabamos de eliminar)

// Atributos como propiedades directas (más común para atributos estándar)
link.href = "https://nueva-url.com";
input.value = "texto precargado";
input.disabled = true;               // deshabilita el input
```

Manipular clases con classList

La propiedad classList expone una API limpia para trabajar con las clases CSS de un elemento sin necesidad de manipular strings:

```
const boton = document.querySelector(".btn");

// Agregar una clase
boton.classList.add("activo");

// Eliminar una clase
boton.classList.remove("activo");

// Toggle: agrega si no está, elimina si está
boton.classList.toggle("activo");

// Verificar si tiene una clase
boton.classList.contains("activo");  // true o false

// Reemplazar una clase por otra
boton.classList.replace("btn-primario", "btn-secundario");

// Agregar múltiples clases a la vez
boton.classList.add("grande", "redondeado", "sombra");
```

```
const toggleBtn = document.querySelector("#toggle-tema");

toggleBtn.addEventListener("click", () => {
  document.body.classList.toggle("dark-mode");
});
```

Estilos inline desde JavaScript

JavaScript puede modificar los estilos CSS de un elemento directamente a través de la propiedad style:

```
const caja = document.querySelector(".caja");

// Las propiedades CSS con guión se escriben en camelCase
caja.style.backgroundColor = "tomato";   // background-color
caja.style.fontSize = "24px";             // font-size
caja.style.borderRadius = "8px";          // border-radius
caja.style.display = "none";              // ocultar elemento
caja.style.display = "";                  // restaurar (elimina el inline style)
```

Crear e insertar elementos en el DOM

Podemos crear elementos HTML nuevos desde JavaScript y agregarlos al DOM:

```
// 1. Crear el elemento
const nuevoParrafo = document.createElement("p");

// 2. Configurarlo
nuevoParrafo.textContent = "Este párrafo fue creado con JavaScript";
nuevoParrafo.classList.add("parrafo-dinamico");
nuevoParrafo.setAttribute("id", "p-nuevo");

// 3. Insertarlo en el DOM
const contenedor = document.querySelector(".contenedor");

contenedor.appendChild(nuevoParrafo);          // al final del contenedor
contenedor.prepend(nuevoParrafo);              // al inicio del contenedor

// Insertar relativo a otro elemento
const referencia = document.querySelector("h2");
referencia.after(nuevoParrafo);                // después del h2
referencia.before(nuevoParrafo);               // antes del h2
```

Eliminar elementos

```
const elemento = document.querySelector(".tarjeta");

// Forma moderna (recomendada)
elemento.remove();

// Forma clásica (a través del padre)
elemento.parentElement.removeChild(elemento);

// Vaciar el contenido de un contenedor
const lista = document.querySelector("ul");
lista.innerHTML = "";        // elimina todo el HTML interno
```

Ejemplo integrador— Lista de tareas dinámica

```html
<!-- HTML base -->
<input type="text" id="input-tarea" placeholder="Nueva tarea...">
<button id="btn-agregar">Agregar</button>
<ul id="lista-tareas"></ul>
```

```javascript
const input  = document.querySelector("#input-tarea");
const boton  = document.querySelector("#btn-agregar");
const lista  = document.querySelector("#lista-tareas");

boton.addEventListener("click", () => {
  const texto = input.value.trim();

  // Validación básica
  if (!texto) return;

  // 1. Crear el li
  const li = document.createElement("li");
  li.textContent = texto;

  // 2. Crear botón de eliminar
  const btnEliminar = document.createElement("button");
  btnEliminar.textContent = "×";
  btnEliminar.addEventListener("click", () => li.remove());

  // 3. Armar y agregar al DOM
  li.appendChild(btnEliminar);
  lista.appendChild(li);

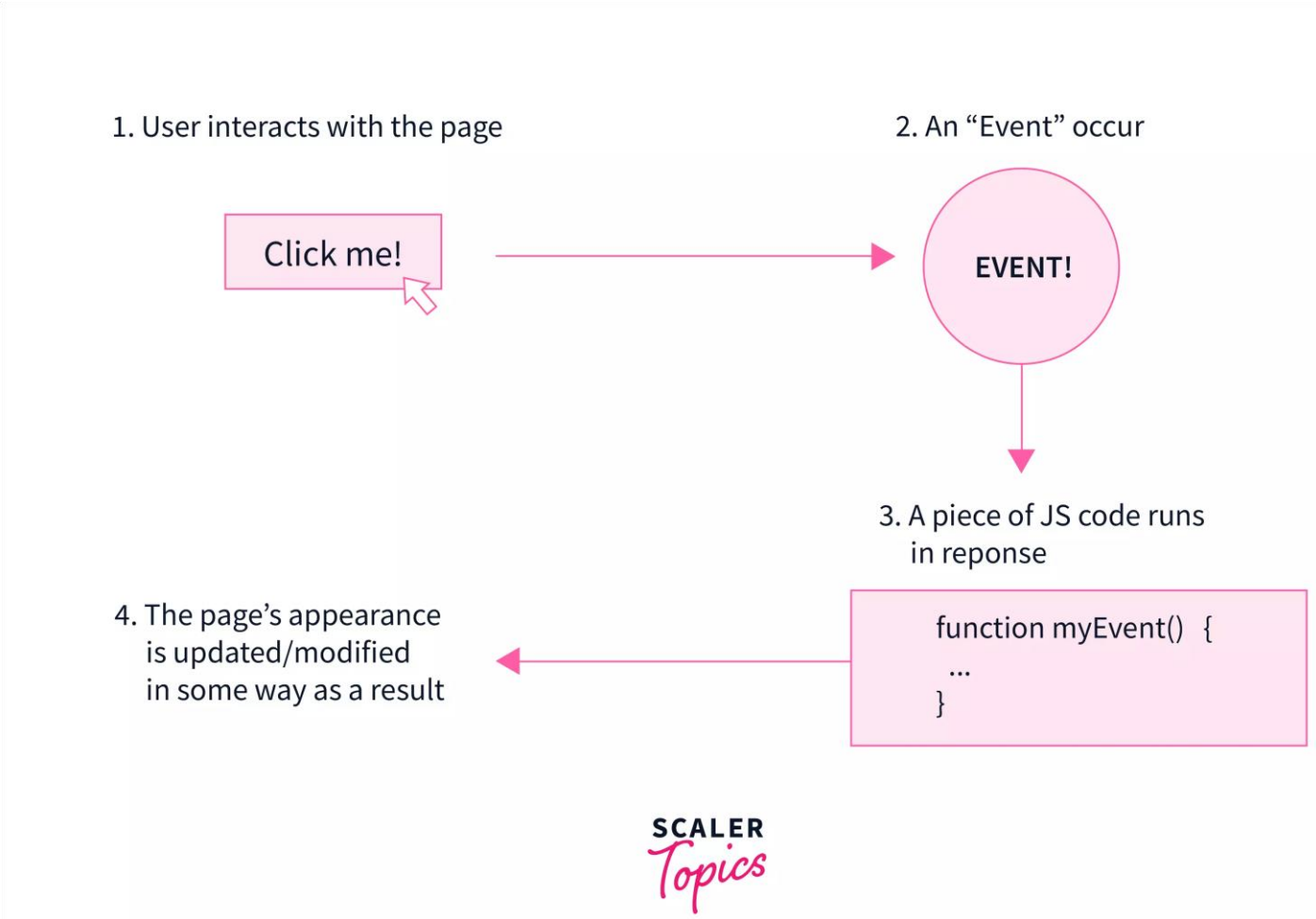
  // 4. Limpiar input
  input.value = "";
  input.focus();
});
```

Resumen Manipulación del DOM

Operación	Herramienta
Leer/escribir texto	textContent, innerHTML
Leer/modificar atributos	getAttribute, setAttribute
Agregar/quitar clases	classList.add/remove/toggle/contains
Modificar estilos	style.propiedad
Crear elementos	document.createElement → configurar → append
Eliminar elementos	elemento.remove()

Separación de responsabilidades: el CSS define cómo se ve, el HTML define la estructura, y el JavaScript define el comportamiento. Cuando JS necesita cambiar la apariencia, lo hace agregando o quitando clases CSS, no escribiendo estilos inline.

Eventos


1. User interacts with the page
2. An “Event” occur
Click me!
EVENT!
3. A piece of JS code runs
in reponse
function myEvent() {
...
}
4. The page’s appearance
is updated/modified
in some way as a result
SCALER
Topics

Eventos — El DOM escucha al usuario

Un **evento** es una señal de que algo ocurrió: el usuario hizo click, movió el mouse, presionó una tecla, se cargó la página, se envió un formulario...

JavaScript puede **escuchar** esos eventos y ejecutar código en respuesta.

Los tres pilares de los eventos:

1. El **elemento** que genera el evento (el target)
2. El **tipo de evento** (ej: "click", "keydown", "submit")
3. El **handler** (la función que se ejecuta cuando ocurre el evento)

```
// Sintaxis básica
elemento.addEventListener("tipoDeEvento", funcionHandler);

// Ejemplo:
const boton = document.querySelector("#btn");

boton.addEventListener("click", function () {
  console.log("¡Botón clickeado!");
});
```

```
// ❌ Inline en HTML: mezcla HTML con JS, difícil de mantener
<button onclick="miFuncion()">Click</button>

// ❌ Propiedad on-: solo admite UN handler por evento
boton.onclick = () => console.log("click");

// ✅ addEventListener: permite múltiples handlers, se puede remover
boton.addEventListener("click", handler);
```

El objeto Event — información sobre lo que ocurrió

Cuando un evento ocurre, el navegador crea automáticamente un objeto Event con información sobre lo que pasó y lo pasa como argumento al handler:

```
boton.addEventListener("click", function (event) {
  console.log(event.type);          // "click"
  console.log(event.target);        // el elemento que fue clickeado
  console.log(event.currentTarget); // el elemento que tiene el listener
  console.log(event.timeStamp);     // cuándo ocurrió (en ms)

  // Coordenadas del mouse
  console.log(event.clientX, event.clientY); // relativas al viewport
  console.log(event.pageX, event.pageY);     // relativas al documento
});

// Con teclado
input.addEventListener("keydown", function (event) {
  console.log(event.key);    // "a", "Enter", "ArrowUp", etc.
  console.log(event.code);   // "KeyA", "Enter", "ArrowUp"
  console.log(event.ctrlKey); // true si se mantuvo Ctrl apretado
});

// Prevenir comportamiento por defecto del navegador
const formulario = document.querySelector("form");
formulario.addEventListener("submit", function (event) {
  event.preventDefault(); // evita que la página se recargue al enviar
  // ahora podemos manejar el submit con JS
});
```

```
// DOMContentLoaded: el más importante para inicializar tu app
document.addEventListener("DOMContentLoaded", () => {
  // Tu código de inicialización acá
  console.log("DOM listo, podemos empezar");
});

// input vs change
input.addEventListener("input", (e) => {
  console.log(e.target.value); // se ejecuta con cada tecla
});

input.addEventListener("change", (e) => {
  console.log(e.target.value); // solo cuando pierde el foco
});
```

Tipos de eventos más comunes

Categoría	Evento	Cúando ocurre
Mouse	click	Click simple
	dblclick	Doble click
	mouseenter	El mouse entra al elemento
	mouseleave	El mouse sale del elemento
	mousemove	El mouse se mueve sobre el elemento
Teclado	keydown	Se presiona una tecla
	keyup	Se suelta una tecla
Formularios	submit	Se envía el formulario
	input	Cambia el valor de un input (en tiempo real)
	change	El valor cambia y el input pierde el foco
	focus	El input recibe el foco
	blur	El input pierde el foco
Documento	DOMContentLoaded	El HTML fue parseado (sin esperar imágenes)
	load	Todo cargó (incluyendo imágenes y CSS)
Ventana	resize	El tamaño de la ventana cambió
	scroll	El usuario scrolleó

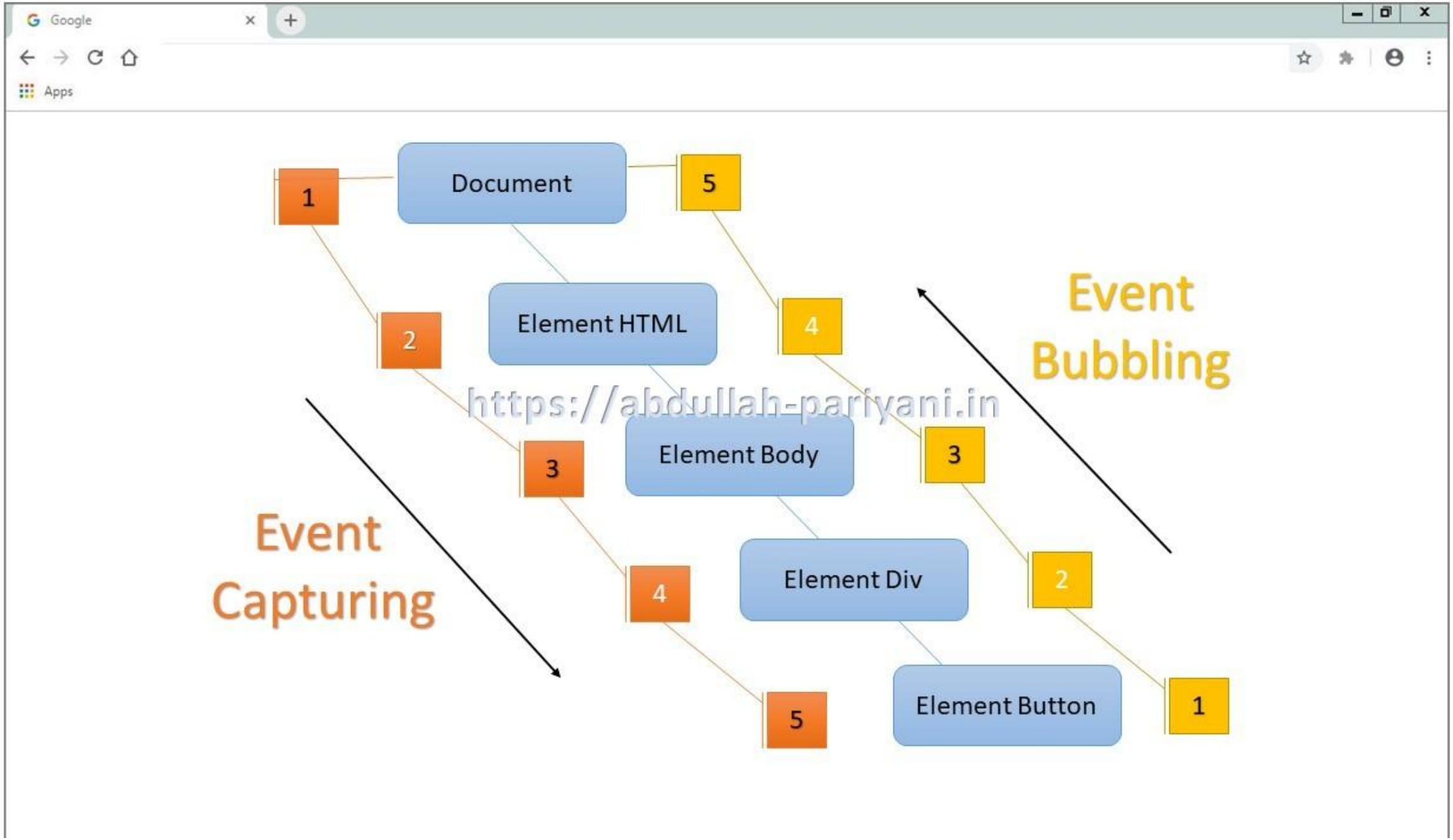
Propagación de eventos: Bubbling y Capturing

Cuando un evento ocurre en un elemento, **no se queda ahí**. Se propaga por el árbol del DOM en dos fases:

1. **Capturing (captura)**: el evento baja desde `document` hasta el elemento objetivo.
2. **Bubbling (burbujeo):** el evento sube desde el elemento objetivo hasta `document`.

document → html → body → div → button (**capturing**: de arriba a abajo)

document ← html ← body ← div ← button (**bubbling**: de abajo a arriba)


Google
Apps
Document
Element HTML
Element Body
Element Div
Element Button
1
2
3
4
5
5
4
3
2
1
Event Capturing
Event Bubbling
https://abdullah-pariyani.in

```
// Por defecto, addEventListener escucha en fase de BUBBLING
div.addEventListener("click", () => console.log("click en div"));
boton.addEventListener("click", () => console.log("click en boton"));

// Al clickear el botón, se imprime:
// "click en boton"
// "click en div"   ← el evento "burbujeó" hasta el div

// Detener el bubbling con stopPropagation
boton.addEventListener("click", (event) => {
  event.stopPropagation(); // el evento no sube más
  console.log("solo este handler se ejecuta");
});

// Para escuchar en fase de capturing (tercer argumento = true)
div.addEventListener("click", handler, true); // poco usado
```

Event Delegation — un patrón más eficiente

En lugar de poner un listener en cada elemento hijo, podemos poner uno solo en el padre y aprovechamos el bubbling para identificar qué hijo fue clickeado (útil en listados grandes).

Problema sin delegation:

Un listener por cada elemento li

```
const items = document.querySelectorAll("li");
items.forEach(item => {
  item.addEventListener("click", () => console.log(item.textContent));
});
```

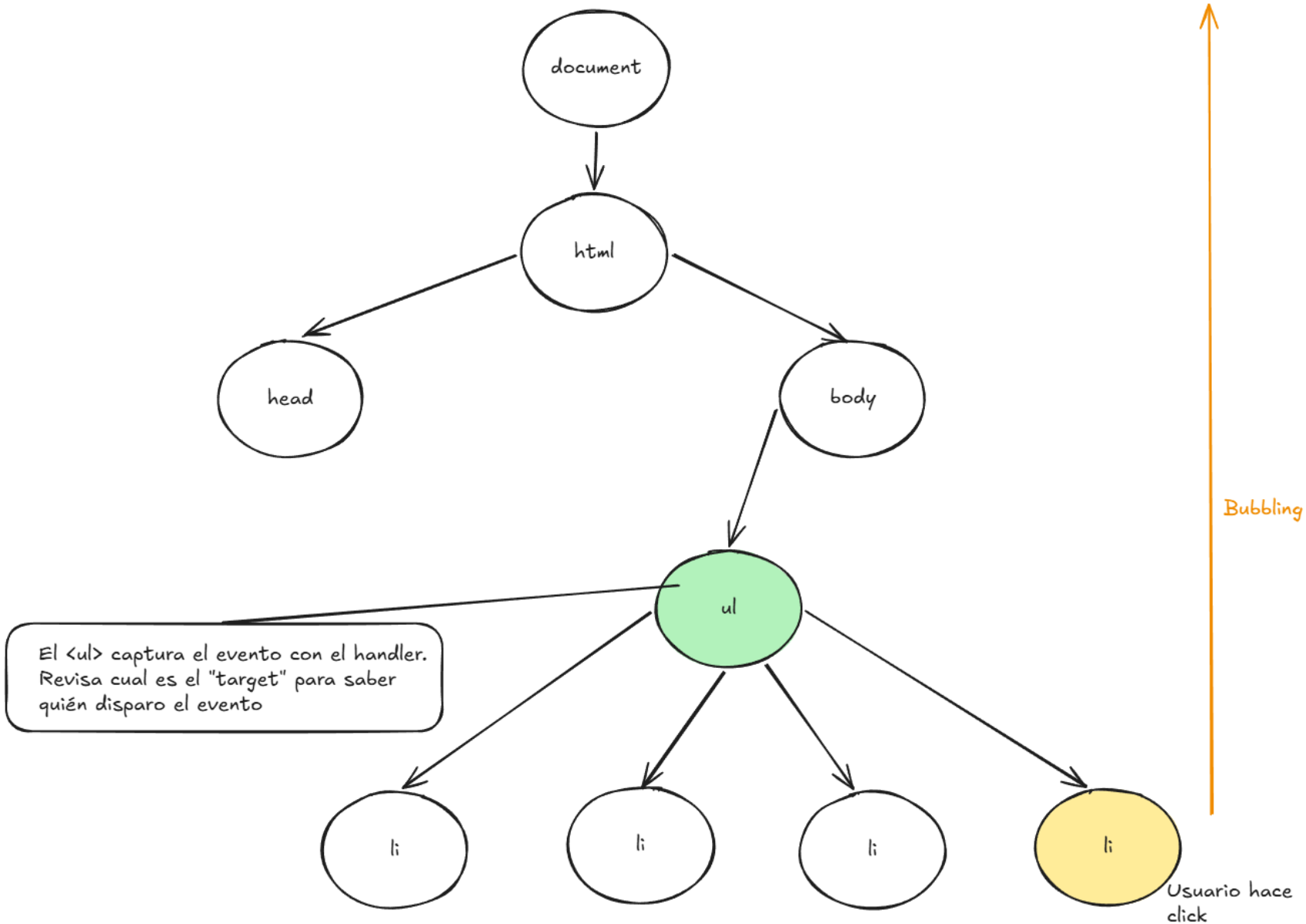
Solución con delegation:

```
// ✅ Un solo listener en el padre
const lista = document.querySelector("ul");

lista.addEventListener("click", (event) => {
  // event.target es el elemento que fue clickeado
  if (event.target.tagName === "LI") {
    console.log(event.target.textContent);
    event.target.classList.toggle("completada");
  }
});

// Esto funciona incluso para li creados DESPUÉS de agregar el listener ✅
const nuevoLi = document.createElement("li");
nuevoLi.textContent = "Nueva tarea";
lista.appendChild(nuevoLi); // este li ya responde al click sin nuevo listener
```

Event delegation es un patrón de performance. Si tenemos una lista con 500 elementos y ponemos un listener en cada uno, tenemos 500 listeners en memoria. Con delegation, solamente uno aunque la lógica del handler aumenta.


document
html
head
body
ul
El <ul> captura el evento con el handler. Revisa cual es el "target" para saber quién disparo el evento
li
li
li
li
Bubbling
Usuario hace click

Trabajar con formularios

```js
const form   = document.querySelector("#form-registro");
const nombre = document.querySelector("#nombre");
const email  = document.querySelector("#email");

// Escuchar el submit y evitar la recarga
form.addEventListener("submit", (event) => {
  event.preventDefault();

  // Leer los valores
  const datos = {
    nombre: nombre.value.trim(),
    email: email.value.trim(),
  };

  // Validación básica
  if (!datos.nombre || !datos.email) {
    mostrarError("Por favor completá todos los campos");
    return;
  }

  console.log("Datos válidos:", datos);
  // Acá iría el fetch a la API
});

// Validación en tiempo real
email.addEventListener("input", (event) => {
  const valor = event.target.value;
  const esValido = valor.includes("@");
  email.classList.toggle("input-error", !esValido);
});
```

```html
<form id="form-registro">
  <input type="text"  id="nombre"   placeholder="Nombre">
  <input type="email" id="email"    placeholder="Email">
  <input type="password" id="pass"  placeholder="Contraseña">
  <button type="submit">Registrarse</button>
</form>
```

DOMContentLoaded y cuándo ejecutar tu código

Uno de los errores más frecuentes es: intentar acceder a elementos del DOM antes de que el HTML haya sido parseado.

```
// ❌ Error clásico: el script está en el <head> y el DOM no existe aún
document.querySelector("#boton"); // null → TypeError al intentar agregar listener

// ✅ Solución 1: escuchar DOMContentLoaded
document.addEventListener("DOMContentLoaded", () => {
  const boton = document.querySelector("#boton");
  // ahora el DOM ya existe
});

// ✅ Solución 2 (más común hoy): poner el script al final del <body>
// <script src="app.js"></script>  ← antes de </body>
```

Ejercicio integrador — Carrito de compras simple

```html
<!-- HTML base: solo la estructura, sin datos hardcodeados -->
<div id="contenedor-productos"></div>

<div id="carrito">
  <h2>Carrito (<span id="cantidad">0</span>)</h2>
  <ul id="lista-carrito"></ul>
  <p>Total: $<span id="total">0</span></p>
</div>
```

```javascript
const productos = [
  { id: 1, nombre: "Remera",   precio: 1500 },
  { id: 2, nombre: "Pantalón", precio: 3200 },
  { id: 3, nombre: "Zapatillas", precio: 7800 },
];

const contenedor   = document.querySelector("#contenedor-productos");
const listaCarrito = document.querySelector("#lista-carrito");
const spanCantidad = document.querySelector("#cantidad");
const spanTotal    = document.querySelector("#total");

let total    = 0;
let cantidad = 0;
```

```
// Construir las tarjetas de productos dinámicamente
productos.forEach((producto) => {
  const div = document.createElement("div");
  div.classList.add("producto");

  const titulo = document.createElement("h3");
  titulo.textContent = producto.nombre;

  const precio = document.createElement("p");
  precio.textContent = `$${producto.precio}`;

  const boton = document.createElement("button");
  boton.textContent = "Agregar al carrito";

  // El closure captura el objeto producto directamente.
  // No necesitamos data-attributes: el botón "ya sabe" qué producto es.
  boton.addEventListener("click", () => {
    agregarAlCarrito(producto);
  });

  div.appendChild(titulo);
  div.appendChild(precio);
  div.appendChild(boton);
  contenedor.appendChild(div);
});
```

```
function agregarAlCarrito(producto) {
  // Crear ítem en el carrito
  const li = document.createElement("li");
  li.textContent = `${producto.nombre} — $${producto.precio}`;
  listaCarrito.appendChild(li);

  // Actualizar totales
  cantidad++;
  total += producto.precio;
  spanCantidad.textContent = cantidad;
  spanTotal.textContent    = total;
}
```

Extender ejercicio:

- Que no permita agregar el mismo producto más de una vez (usar .find).
- Agregar un botón para eliminar ítems del carrito y que el total se actualice.
- Mostrar un mensaje "El carrito está vacío" cuando no hay ítems.


UTN
FACULTAD REGIONAL
RAFAELA